



Unidad IV. Desarrollo de Aplicaciones web dinámicas	ACTIVIDAD: DSW-A13. Despliegue de aplicaciones web (Node.js) en Azure.
TIPO: Actividad individual	

OBJETIVO

El estudiante desplegará una aplicación web desarrollada con Node.js, Express, EJS y MySQL sobre una máquina virtual Linux en Microsoft Azure, configurando la infraestructura necesaria, instalando los componentes del entorno de ejecución, aplicando controles básicos de seguridad y verificando el funcionamiento de la aplicación mediante acceso remoto desde Internet.

DESCRIPCIÓN

En esta práctica, el estudiante utilizará su cuenta institucional de Azure para crear y configurar una máquina virtual Ubuntu Linux en la nube. Posteriormente instalará Node.js, MySQL y las dependencias necesarias para ejecutar la aplicación desarrollada en la actividad DSW-A12.

La práctica se divide en dos partes:

- Parte A. Infraestructura: Creación de la máquina virtual, configuración de red, instalación del sistema y aseguramiento básico del servidor.
- Parte B. Despliegue: Transferencia del proyecto, configuración del entorno, creación de la base de datos y publicación de la aplicación.

Al finalizar, el sistema deberá estar accesible mediante la dirección IP pública proporcionada por Azure.

MATERIAL Y EQUIPO REQUERIDOS

- a) Cuenta institucional de Microsoft Azure con crédito educativo.
- b) Aplicación desarrollada en la práctica DSW-A12.
- c) Cliente SSH:
 - Terminal (macOS/Linux),
 - PowerShell o Windows Terminal,
 - PuTTY (opcional).
- d) Navegador web actualizado.
- e) Visual Studio Code (opcional).
- f) Conexión estable a Internet.

PROCEDIMIENTO

Parte A. Infraestructura: Creación de la máquina virtual, configuración de red, instalación del sistema y aseguramiento básico del servidor

Paso 1 – Ingresar al portal de Azure

1. Abre el navegador web.
2. Ingresa al portal de Azure:

`https://portal.azure.com`

3. Inicia sesión con tu cuenta institucional.



4. Verifica que tengas una suscripción activa.

Paso 2 – Crear un grupo de recursos

1. En la barra de búsqueda superior escribe:

Resource groups o Grupos de recursos

2. Selecciona Resource groups.
3. Haz clic en Create.
4. Configura:

Campo	Valor sugerido
Subscription	Tu suscripción activa
Resource group	rg-dsw-a13-nombreapellido (Si es en parejas, los primeros nombres de ambos)
Region	East US, Central US o la región disponible más cercana

5. Haz clic en Review + create.
6. Después haz clic en Create.

El grupo de recursos permite administrar y eliminar todos los elementos de la práctica en conjunto.



Microsoft Azure

Buscar recursos, servicios y documentos (G+ /)

Inicio > Resource Manager | Grupos de recursos

Crear un grupo de recursos

Datos básicos

Etiquetas

Revisar y crear

Grupo de recursos - Contenedor que incluye los recursos relacionados para una solución de Azure. El grupo de recursos puede contener todos los recursos de la solución o solamente los recursos que quiere administrar en grupo. Debe decidir cómo quiere asignar los recursos a los grupos de recursos según lo que resulte más pertinente para su organización. [Más información](#)

Suscripción * ⓘ

Suscripción de Azure Erika

Nombre del grupo de recursos * ⓘ

rg-dsw-a13-ErikaMeneses

Región * ⓘ

(US) East US

Anterior

Siguiente

Revisar y crear

+ Crear

⚙ Administrar vista

↻ Actualizar

↓ Exportar a CSV

🔗 Abrir consulta

🏷 Asignar etiquetas

...

🔍 Filtrar por cualquier ...

Suscripción es igual que **todo**

Ubicación es igual que **todo**

✕

+ Agregar filtro

☐	Nombre ↑	Suscripción	Ubicación
☐	 rg-dsw-a13-ErikaMeneses	... Suscripción de Azure Erika	East US



Paso 3 – Crear una máquina virtual Linux

1. En el portal de Azure, busca:

Virtual machines o Máquinas virtuales

2. Selecciona Virtual machines.
3. Haz clic en Create → Azure virtual machine.

Paso 4 – Configurar datos básicos de la máquina virtual

En la pestaña Basics, configura los siguientes datos:

Campo	Valor sugerido
Subscription	Tu suscripción
Resource group	rg-dsw-a13-nombreapellido
Virtual machine name	vm-dsw-a13-nombreapellido
Region	Misma región del grupo de recursos
Availability options	No infrastructure redundancy required
Security type	Standard
Image	Ubuntu Server 22.04 LTS o Ubuntu Server 24.04 LTS
VM architecture	x64
Size	B1s o la opción gratuita/más económica disponible

Microsoft Azure

Buscar recursos, servicios y documentos (G+/)

Copilot

ermeneses@uv.mx
UNIVERSIDAD VERACRUZANA (U...

Inicio > Infraestructura de proceso | Máquinas virtuales

Crear una máquina virtual

Datos básicos

Discos

Redes

Administración

Supervisión

Opciones avanzadas

Etiquetas

Revisar y crear

Cree una máquina virtual que ejecuta Linux o Windows. Seleccione una imagen de Azure Marketplace o use una imagen personalizada propia. Complete la pestaña Conceptos básicos y, después, use Revisar y crear para aprovisionar una máquina virtual con parámetros predeterminados o bien revise cada una de las pestañas para personalizar la configuración.

[Más información](#)

Detalles del proyecto

Seleccione la suscripción para administrar recursos implementados y los costes. Use los grupos de recursos como carpetas para organizar y administrar todos los recursos.

Suscripción * ⓘ Suscripción de Azure Erika

Grupo de recursos * ⓘ rg-dsw-a13-ErikaMeneses
[Crear nuevo](#)

Detalles de instancia

Nombre de máquina virtual * ⓘ vm-dsw-a13-ErikaMeneses

Costos mensuales estimados

Enviar comentarios sobre su experiencia de estimación

Básico

\$15.11

Máquina virtual

\$15.11

Imagen

Ubuntu Server 24.04 LTS

\$0.00

Tamaño

Standard_B1ms

\$15.11

> Discos

\$5.28

> Redes

\$0.00

Costo mensual estimado

\$20.39

< Anterior

Siguiente: Discos >

Revisar y crear

Enviar comentarios



Región *	(US) East US
	Implementación en una zona extendida de Azure
Opciones de disponibilidad	No se requiere redundancia de la infraestructura
Tipo de seguridad	Máquinas virtuales de inicio seguro
	Configurar características de seguridad
Imagen *	Ubuntu Server 24.04 LTS - x64 gen. 2
	Ver todas las imágenes Configurar la generación de máquinas virtuales
Arquitectura de VM	☐ Arm64 ☒ x64
Ejecución de Azure Spot con descuento	☐
Tamaño *	Standard_B1ms - 1 vcpu, 2 GiB de memoria (USD 15.11)
	Ver todos los tamaños

Paso 5 – Configurar la cuenta de administrador

En el apartado Administrator account, configura:

Campo	Valor
Authentication type	SSH public key
Username	azureuser o un usuario personalizado (anota el usuario ingresado)
SSH public key source	Generate new key pair
Type	Formato RSA SSH
Key pair name	key-dsw-a13-nombreapellido

Después, al crear la máquina virtual, Azure descargará un archivo .pem.

Guarda ese archivo en un lugar seguro, porque será necesario para conectarte por SSH y no podrás generarlo de nuevo posteriormente.

Paso 6 – Configurar reglas de entrada en Azure

En la sección Inbound port rules, selecciona inicialmente:

Allow selected ports

Activa:



UNIVERSIDAD VERACRUZANA. Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

EE. Desarrollo de Sistemas Web

SSH (22)
HTTP (80)

Posteriormente, si se requiere exponer temporalmente la aplicación Node.js directamente, se agregará el puerto 3000.

En Azure, el tráfico hacia una VM se controla mediante reglas de seguridad de red o NSG, que permiten o deniegan tráfico entrante y saliente por puerto, protocolo y origen.

Habilitar hibernación ⓘ

☐

Actualmente, Hibernar no admite el inicio de confianza y las máquinas virtuales confidenciales para imágenes de Linux. [Más información](#)

Cuenta de administrador

Tipo de autenticación ⓘ

☒ Clave pública SSH

☐ Contraseña

Ahora, Azure genera automáticamente un par de claves SSH y le permite almacenarlo para usarlo en el futuro. Es una forma rápida, sencilla y segura de conectarse a la máquina virtual.

Nombre de usuario * ⓘ

azureuser ✓

Origen de clave pública SSH

Generar un par de claves nuevo ▾

Tipo de clave SSH

☒ Formato RSA SSH

☐ Formato Ed25519 SSH

Ed25519 proporciona un nivel de seguridad fijo de no más de 128 bits para claves

Nombre de par de claves *

vm-dsw-a13-ErikaMeneses_key ✓

Reglas de puerto de entrada

Seleccione los puertos de red de máquina virtual que son accesibles desde la red Internet pública. Puede especificar acceso de red más limitado o granular en la pestaña Red.

Puertos de entrada públicos * ⓘ

☐ Ninguno

☒ Permitir los puertos seleccionados

Seleccionar puertos de entrada *

HTTP (80), SSH (22) ▾

⚠ Esto permitirá que todas las direcciones IP accedan a la máquina virtual. Esto solo se recomienda para las pruebas. Use los controles avanzados de la pestaña Redes a fin de crear reglas para limitar el tráfico entrante a las direcciones IP conocidas.

< Anterior

Siguiente: Discos >

Revisar y crear



Paso 7 – Revisar y crear la máquina virtual

1. Haz clic en Review + create.
2. Revisa la configuración.
3. Haz clic en Create.
4. Si Azure genera una llave SSH, **descarga el archivo .pem y guárdalo en un lugar seguro.**
5. Espera a que finalice el despliegue.

The screenshot shows the Microsoft Azure portal interface. At the top, there's a search bar and navigation icons. The main heading is "CreateVm-canonical.ubuntu-24_04-lts-server-20260520143059 | Información general". Below this, there's a status bar with icons for "Eliminar", "Cancelar", "Volver a implementar", "Descargar", and "Actualizar". The left sidebar shows "Información general" selected, with sub-items "Entradas", "Salidas", and "Plantilla". The main content area displays a green checkmark and the text "Se completó la implementación". Below this, it lists deployment details: "Nombre de implementación : CreateVm-canonical.ubu...", "Hora de inicio : 20/5/2026, 2:55:29 p.m.", "Suscripción : Suscripción de Azure Erika", "Id. de correlación : c493ce35-4c6c-414d-80d2-e6a...", and "Grupo de recursos : rg-dsw-a13-ErikaMeneses". A section titled "Detalles de implementación" is expanded, showing a table of resources.

Recurso	Tipo	Estado	Detalles de la o
vm-dsw-a13-Erika	Microsoft.Compute/virtualMach	OK	Detalles de la o
vm-dsw-a13-erika	Microsoft.Network/networkInter	OK	Detalles de la o
vm-dsw-a13-Erika	Dirección IP pública	OK	Detalles de la o
vm-dsw-a13-Erika	Red virtual	OK	Detalles de la o
vm-dsw-a13-Erika	Grupo de seguridad de red	OK	Detalles de la o

Paso 8 – Obtener la IP pública de la máquina virtual

1. Entra al recurso de la máquina virtual.
2. En la vista general (Overview), localiza:

Public IP address

3. Copia esa dirección IP.

Ejemplo:

20.120.55.10



Paso 9 – Conectarse a la máquina virtual por SSH

Desde la terminal de tu equipo local, ubícate en la carpeta donde guardaste la llave .pem.

En macOS o Linux:

```
chmod 400 key-dsw-a13-nombreapellido.pem  
ssh -i key-dsw-a13-nombreapellido.pem azureuser@IP_PUBLICA
```

Ejemplo:

```
ssh -i key-dsw-a13-erikameneses.pem azureuser@20.120.55.10
```

En Windows PowerShell:

```
ssh -i .\key-dsw-a13-nombreapellido.pem azureuser@IP_PUBLICA
```

Si la conexión es exitosa, verás la terminal de Ubuntu.

Si la red no permite conectar vía SSH, hazlo mediante el servicio Azure Bastion:

<https://learn.microsoft.com/en-us/azure/bastion/bastion-connect-vm-ssh-linux?tabs=entra-id%2Cportal>

Paso 10 – Actualizar paquetes del sistema

Dentro de la máquina virtual, ejecuta:

```
sudo apt update  
sudo apt upgrade -y
```

Este paso actualiza el índice de paquetes y aplica actualizaciones disponibles.

Paso 11 – Instalar herramientas básicas

Instala utilidades necesarias para administrar el servidor:

```
sudo apt install -y curl wget git unzip ufw build-essential
```

Paso 12 – Configurar firewall UFW en Ubuntu

Primero permite SSH para evitar perder la conexión:

```
sudo ufw allow OpenSSH
```

Permite tráfico HTTP:



```
sudo ufw allow 80/tcp
```

Permite temporalmente el puerto de desarrollo de Node.js:

```
sudo ufw allow 3000/tcp
```

Activa UFW:

```
sudo ufw enable
```

Verifica el estado:

```
sudo ufw status numbered
```

```
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo ufw allow OpenSSH
Rules updated
Rules updated (v6)
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo ufw allow 80/tcp
Rules updated
Rules updated (v6)
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo ufw allow 3000/tcp
Rules updated
Rules updated (v6)
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo ufw enable
[Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo ufw status numbered
Status: active
```

To	Action	From
---	-----	----
[1] OpenSSH	ALLOW IN	Anywhere
[2] 80/tcp	ALLOW IN	Anywhere
[3] 3000/tcp	ALLOW IN	Anywhere
[4] OpenSSH (v6)	ALLOW IN	Anywhere (v6)
[5] 80/tcp (v6)	ALLOW IN	Anywhere (v6)
[6] 3000/tcp (v6)	ALLOW IN	Anywhere (v6)

Ubuntu usa UFW como herramienta sencilla para configurar reglas de firewall; por defecto se encuentra deshabilitado.

Es importante permitir SSH antes de habilitar UFW para no perder el acceso remoto al servidor.

Paso 13 – Verificar conectividad básica del servidor

Desde la VM ejecuta:

```
hostname
ip addr
whoami
```



```
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ hostname
vm-dsw-a13-2-ErikaMeneses
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:0d:3a:55:e4:bb brd ff:ff:ff:ff:ff:ff
    inet 10.0.0.4/24 metric 100 brd 10.0.0.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::20d:3aff:fe55:e4bb/64 scope link
        valid_lft forever preferred_lft forever
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ whoami
azureuser
```

También verifica que el servidor responde a comandos de red:

```
curl ifconfig.me
```

El resultado debe mostrar la IP pública o una IP de salida asociada a Azure.

```
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ curl ifconfig.me
104.211.57.187azureuser@vm-dsw-a13-2-ErikaMeneses:~$
```

Paso 14 – Instalar Node.js y npm

Para instalar Node.js desde los repositorios de Ubuntu:

```
sudo apt install -y nodejs npm
```

Verifica:

```
node -v
npm -v
```

```
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ node -v
v18.19.1
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ npm -v
9.2.0
```

Si la versión instalada es antigua, en la Parte B se puede actualizar usando NodeSource o nvm, pero para esta primera parte basta con validar que Node.js y npm funcionan.

Paso 15 – Instalar MySQL Server

Instala MySQL:

```
sudo apt install -y mysql-server
```

Verifica el estado del servicio:



```
sudo systemctl status mysql
```

```
azureuser@vm-dsw-a13-2-ErikaMeneses:~$ sudo systemctl status mysql
● mysql.service - MySQL Community Server
   Loaded: loaded (/usr/lib/systemd/system/mysql.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-05-21 17:57:17 UTC; 24s ago
     Process: 9510 ExecStartPre=/usr/share/mysql/mysql-systemd-start pre (code=exited, status=0/SUCCESS)
    Main PID: 9519 (mysqld)
      Status: "Server is operational"
        Tasks: 38 (limit: 2245)
       Memory: 351.6M (peak: 381.0M)
          CPU: 870ms
      CGroup: /system.slice/mysql.service
              └─9519 /usr/sbin/mysqld

May 21 17:57:16 vm-dsw-a13-2-ErikaMeneses systemd[1]: Starting mysql.service - MySQL Community Server...
May 21 17:57:17 vm-dsw-a13-2-ErikaMeneses systemd[1]: Started mysql.service - MySQL Community Server.
```

Si no está activo, inicia el servicio:

```
sudo systemctl start mysql
```

Habilita MySQL al arranque:

```
sudo systemctl enable mysql
```

Accede a MySQL:

```
sudo mysql
```

Si el acceso es exitoso, aparecerá el prompt de MySQL:

```
mysql>
```

Dentro de MySQL ejecuta:

```
ALTER USER 'root'@'localhost'
IDENTIFIED WITH mysql_native_password BY 'TuPasswordSegura123!';
```

Sustituye TuPasswordSegura123! por una contraseña segura.

Aplica los cambios:

```
FLUSH PRIVILEGES;
```

Verifica el usuario actual:

```
SELECT user, host FROM mysql.user;
```



Dentro de MySQL, verifica bases de datos:

```
SHOW DATABASES;
```

Salir:

```
EXIT;
```

```
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.01 sec)

mysql> SELECT user, host FROM mysql.user;
+-----+-----+
| user | host |
+-----+-----+
| debian-sys-maint | localhost |
| mysql.infoschema | localhost |
| mysql.session | localhost |
| mysql.sys | localhost |
| root | localhost |
+-----+-----+
5 rows in set (0.00 sec)

mysql> exit
```

Ahora prueba conectarte usando:

```
mysql -u root -p
```

El sistema solicitará la contraseña configurada previamente.

Si la autenticación es exitosa, aparecerá nuevamente el prompt:

```
mysql>EXIT;
```

Ejecuta el asistente de seguridad:



```
sudo mysql_secure_installation
```

El asistente solicitará varias configuraciones.

Se recomienda responder:

Pregunta	Respuesta sugerida
VALIDATE PASSWORD COMPONENT	Yes
Nivel de validación	MEDIUM
Remove anonymous users	Yes
Disallow root login remotely	Yes
Remove test database	Yes
Reload privilege tables	Yes

Este procedimiento elimina configuraciones inseguras predeterminadas y fortalece la instalación inicial de MySQL.

Paso 16 – Crear una carpeta de trabajo para aplicaciones web

Crea una carpeta para alojar el proyecto:

```
sudo mkdir -p /var/www/dsw-a13
```

Asigna permisos al usuario actual:

```
sudo chown -R $USER:$USER /var/www/dsw-a13
```

Entra a la carpeta:

```
cd /var/www/dsw-a13
```

Paso 17 – Crear una aplicación mínima de prueba en Node.js

Para validar que Node.js funciona antes de desplegar la aplicación real:

```
nano prueba-node.js
```

Agrega:

```
const http = require("http");  
  
const server = http.createServer((req, res) => {
```



```
res.end("Servidor Node.js funcionando en Azure");  
});  
  
server.listen(3000, "0.0.0.0", () => {  
  console.log("Servidor escuchando en puerto 3000");  
});
```

Guarda con:

```
Ctrl + O  
Enter  
Ctrl + X
```

Paso 18 – Probar la aplicación desde el navegador

Para probar la aplicación desde el navegador primero abre el puerto 3000 en Azure

Para permitir el acceso externo a la aplicación Node.js, es necesario crear una regla de entrada en el grupo de seguridad de red (NSG) asociado a la máquina virtual.

1. En el portal de Azure, abre la máquina virtual.
2. En el menú lateral selecciona:

Networking

3. En la sección:

Inbound port rules

haz clic en:

Add inbound port rule



UNIVERSIDAD VERACRUZANA. Licenciatura en Ingeniería de Ciberseguridad e Infraestructura de Cómputo

EE. Desarrollo de Sistemas Web

Inicio > vm-dsw-a13-2-ErikaMeneses

vm-dsw-a13-2-ErikaMeneses | Configuración de red ☆ ... ¿Cuáles son los requisitos para conectar o desasociar una interfaz de red? +2

Reglas de seguridad ... : 0 (Configurar)

Reglas de seguridad v... : 0

Reglas

Contraer todo

Grupo de seguridad de red **vm-dsw-a13-2-ErikaMeneses-nsg** (conectado a networkInterface: **vm-dsw-a13-2-erikameneses468**)

Afecta a 0 subredes, 1 interfaces de red

Origen == todo

Destino == todo

Protocolo == todo

Acción == todo

Puerto == todo

Crear ACL del puerto

Regla de puerto de entrada

Regla de puerto de salida

Prioridad	Nombre	Puerto	Protocolo	Origen	Destino	Acción
Reglas de puerto de entrada (5)						
300	SSH	22	TCP	Cualquiera	Cualquiera	Allow
320	HTTP	80	TCP	Cualquiera	Cualquiera	Allow
65000	AllowVnetInBound	Cualquiera	Cualquiera	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerInBou...	Cualquiera	Cualquiera	AzureLoadBalancer	Cualquiera	Allow
65500	DenyAllInBound	Cualquiera	Cualquiera	Cualquiera	Cualquiera	Deny
Reglas de puerto de salida (3)						

4. Configura la regla:

Campo	Valor
Source	Any (o My IP para mayor seguridad)
Source port ranges	*
Destination	Any
Service	Custom
Destination port ranges	3000
Protocol	TCP
Action	Allow
Priority	310
Name	Allow-Node-3000

5. Haz clic en Add.

Reglas de puerto de entrada (6)						
300	SSH	22	TCP	Cualquiera	Cualquiera	Allow
310	Allow-Node-3000	3000	TCP	148.226.80.25	Cualquiera	Allow
320	HTTP	80	TCP	Cualquiera	Cualquiera	Allow
65000	AllowVnetInBound	Cualquiera	Cualquiera	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerInBound	Cualquiera	Cualquiera	AzureLoadBalancer	Cualquiera	Allow
65500	DenyAllInBound	Cualquiera	Cualquiera	Cualquiera	Cualquiera	Deny

Ahora abre el puerto 3000 en UFW (Ubuntu), si aún no lo tienes:



Dentro de la máquina virtual, ejecuta:

```
sudo ufw allow 3000/tcp
```

Verifica las reglas activas:

```
sudo ufw status
```

Debe aparecer algo similar a:

3000/tcp	ALLOW	Anywhere
----------	-------	----------

```
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ sudo ufw status
Status: active

To Action From
--
OpenSSH ALLOW Anywhere
80/tcp ALLOW Anywhere
3000/tcp ALLOW Anywhere
OpenSSH (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
3000/tcp (v6) ALLOW Anywhere (v6)
```

Ejecuta:

```
node prueba-node.js
```

```
[azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ node prueba-node.js
Servidor escuchando en puerto 3000
```

Desde tu equipo local abre:

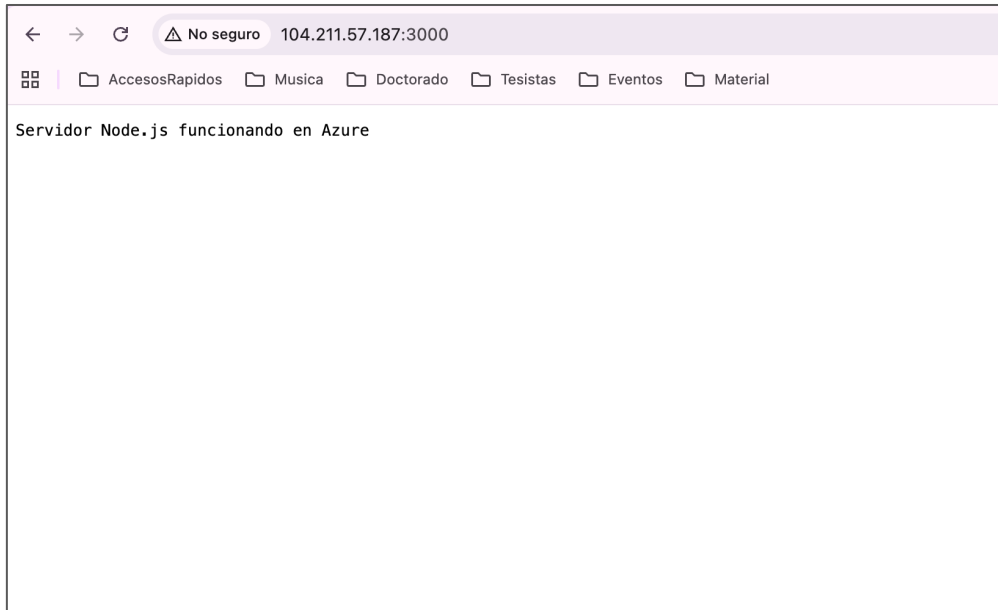
```
http://IP_PUBLICA:3000
```

Ejemplo:

```
http://20.120.55.10:3000
```

Debe aparecer:

Servidor Node.js funcionando en Azure



Si no aparece, revisa:

1. Que el proceso Node siga activo.
2. Que UFW permita el puerto 3000.
3. Que Azure tenga una regla de entrada para el puerto 3000 en el NSG.

Paso 19 – Detener el servidor de prueba

Regresa a la terminal donde está corriendo Node.js y presiona:

```
Ctrl + C
```

Después elimina el archivo de prueba si lo deseas:

```
rm prueba-node.js
```

Paso 20 – Verificar estado final del servidor

Ejecuta:

```
node -v  
npm -v  
mysql --version
```



```
sudo ufw status  
git --version
```

El servidor debe contar con:

- Ubuntu actualizado.
- Node.js instalado.
- npm instalado.
- MySQL instalado y activo.
- Git instalado.
- UFW configurado.
- Puertos 22, 80 y 3000 permitidos según corresponda.

Parte B. Despliegue: transferencia del proyecto, configuración del entorno, creación de la base de datos y publicación de la aplicación

Paso 1 – Conectarse por SSH a la máquina virtual

Desde tu equipo local, ejecuta:

```
ssh -i ruta/llave.pem azureuser@IP_PUBLICA
```

Ejemplo:

```
ssh -i ~/Downloads/vm-dsw-a13.pem azureuser@20.120.55.10
```

O conéctate a través del servicio Bastión si la red no te permite conectarte mediante SSH

Paso 2 – Crear carpeta para la aplicación si aún no la tienes

Dentro de la VM:

```
sudo mkdir -p /var/www/dsw-a13  
sudo chown -R $USER:$USER /var/www/dsw-a13  
cd /var/www/dsw-a13
```

Paso 3 – Transferir el proyecto a la VM

Opción A. Clonar desde GitHub

```
git clone URL_DEL_REPOSITORIO .
```

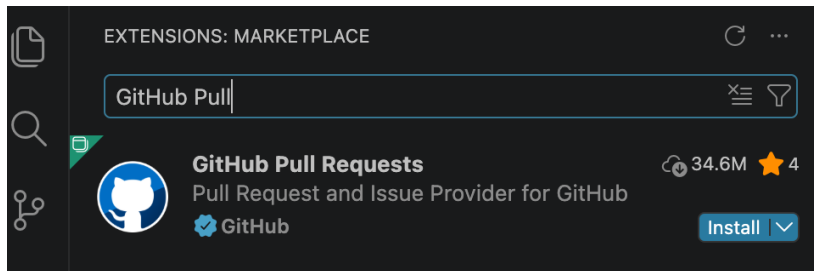


Ejemplo:

```
git clone https://github.com/usuario/proyecto-dsw.git .
```

Subir el proyecto a GitHub usando Visual Studio Code

Instala en Visual Studio Code la extensión GitHub Pull Requests



Paso 1 – Abrir el proyecto en VS Code

1. Abre Visual Studio Code.
2. Selecciona:

File → Open Folder

3. Abre la carpeta del proyecto:

DSW-A12

Paso 2 – Crear el archivo .gitignore

1. En el explorador lateral, haz clic derecho sobre la raíz del proyecto.
2. Selecciona:

New File

3. Nombra el archivo:

.gitignore

4. Agrega el siguiente contenido:

```
node_modules/  
.env  
.DS_Store
```

Guarda el archivo



```
DSW-A12 - nano .gitignore - 195x49
UW PICO 5.09
node_modules/
.env
.DS_Store
```

Paso 3 - Inicializa Git

1. Abre la terminal integrada:

Terminal → New Terminal

2. Ejecuta:

```
git init
macerika@MacBook-de-Mac DSW-A12 % git init
Initialized empty Git repository in /Users/macerika/Library/CloudStorage/OneDrive-UniversidadVeracruzana/Mis documentos/2026/Febrero-Julio/DSW/Actividades/DSW-A12/.git/
```

VS Code detectará automáticamente el repositorio Git.

Paso 4 – Agregar archivos al repositorio

En la terminal ejecuta:

```
git add .
```

Paso 5 – Crear el primer commit

Ejecuta:

```
git commit -m "Proyecto MVC Node.js inicial"
```



```
macerika@MacBook-de-Mac DSW-A12 % git add .
macerika@MacBook-de-Mac DSW-A12 % git commit -m "Proyecto MVC Node.js inicial"
[main (root-commit) d4ca43b] Proyecto MVC Node.js inicial
Committer: Mac de Erika <macerika@MacBook-de-Mac.local>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

12 files changed, 1531 insertions(+)
create mode 100644 .gitignore
create mode 100644 bd.sql
create mode 100644 controllers/home.js
create mode 100644 data/datacontext.js
create mode 100644 index.js
create mode 100644 middlewares/errorhandler.js
create mode 100644 models/alumno.js
create mode 100644 package-lock.json
create mode 100644 package.json
create mode 100644 routes/home.js
create mode 100644 views/home/_partialcard.ejs
create mode 100644 views/home/index.ejs
```

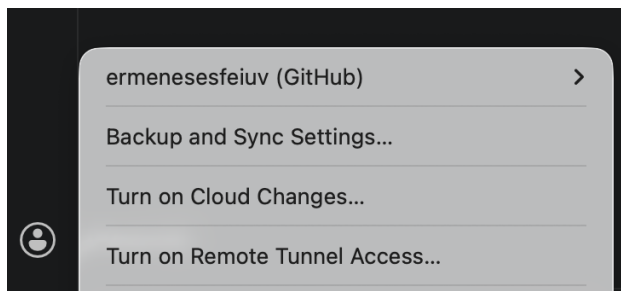
Paso 6 – Iniciar sesión en GitHub desde VS Code

1. En la esquina inferior izquierda de VS Code haz clic en el icono de cuenta.
2. Selecciona:

Sign in with GitHub

3. El navegador se abrirá.
4. Autoriza Visual Studio Code.
5. Regresa a VS Code.

Debe aparecer:



Paso 7 – Publica el proyecto



Después del commit, VS Code mostrará un botón:

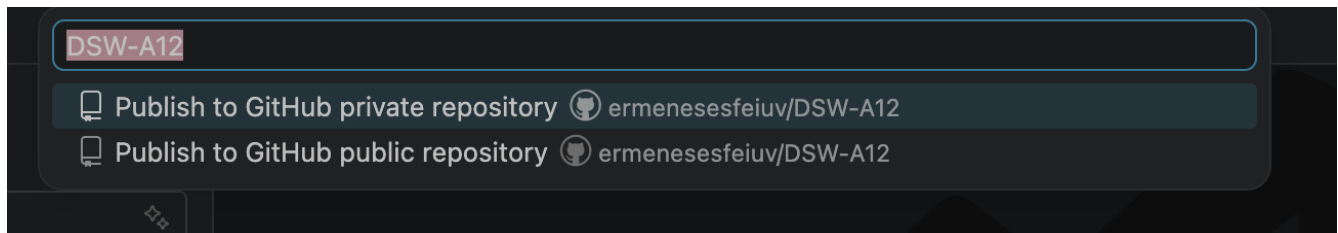
Publish Branch

o:

Publish to GitHub

Haz clic ahí.

Si no aparece el botón, presiona Ctrl + Shift + P y escribe Publish to GitHub



Elige private repository.

Si solicita Configurar el repositorio

Opción	Valor recomendado
Repository name	DSW-A13
Visibility	Private

Selecciona:

Publish to GitHub private repository

Paso 8 – Esperar sincronización

VS Code automáticamente:

- Crea el repositorio
- Configura origin
- Hace push
- Sincroniza el proyecto

Paso 9 – Verificar en GitHub

Abre:



<https://github.com>

En tu perfil deberá aparecer el repositorio.

ermenesesfeiu / DSW-A13

<> Code Issues Pull requests Actions Projects Security and quality Insights Settings

DSW-A13 Private Watch 0

main 1 Branch 0 Tags Go to file T Add file <> Code

Mac de Erika and Mac de Erika Proyecto MVC Node.js inicial d4ca43b · 1 hour ago 1 Commit

controllers	Proyecto MVC Node.js inicial	1 hour ago
data	Proyecto MVC Node.js inicial	1 hour ago
middlewares	Proyecto MVC Node.js inicial	1 hour ago
models	Proyecto MVC Node.js inicial	1 hour ago
routes	Proyecto MVC Node.js inicial	1 hour ago
views/home	Proyecto MVC Node.js inicial	1 hour ago
.gitignore	Proyecto MVC Node.js inicial	1 hour ago
bd.sql	Proyecto MVC Node.js inicial	1 hour ago
index.js	Proyecto MVC Node.js inicial	1 hour ago

Paso 10 – Verificar .gitignore

En GitHub NO deben aparecer:

```
node_modules
.env
```

Eso confirma que .gitignore funciona correctamente.

Paso 11 - Clonar el proyecto a Azure:

1. Haz el repositorio público en GitHub

En GitHub:

Repositorio → Settings → General



Abajo:

Danger Zone → Change repository visibility

Selecciona:

Public

Danger Zone

Change repository visibility This repository is currently private.	Change visibility
Disable branch protection rules Disable branch protection rules enforcement and APIs	Change to public Disable branch protection rules
Transfer ownership Transfer this repository to another user or to an organization where you have the ability to create repositories.	Transfer
Archive this repository Mark this repository as archived and read-only.	Archive this repository
Delete this repository Once you delete a repository, there is no going back. Please be certain.	Delete this repository

2. Ahora en Azure:

```
cd /var/www/dsw-a13
git clone https://github.com/USUARIO/NOMBRE_REPOSITORIO.git .
```

```
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ git clone https://github.com/ermenesesfeiu/Dsw-A13.git .
Cloning into '.'...
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (14/14), done.
remote: Total 21 (delta 0), reused 21 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (21/21), 15.99 KiB | 8.00 MiB/s, done.
azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ ls
bd.sql  controllers  data  index.js  middlewares  models  package-lock.json  package.json  routes  views
```

Vuelve el repositorio a privado.

En producción normalmente los repositorios serían privados y usarían autenticación.

Opción B. Copiar con scp

Desde tu equipo local:



```
scp -i ~/Downloads/vm-dsw-a13.pem -r ~/Downloads/DSW-A12/*  
azureuser@104.211.57.187:/var/www/dsw-a13
```

Conéctate de nuevo al servidor y verifica:

```
cd /var/www/dsw-a13  
ls
```

Paso 4 – Instalar dependencias del proyecto

Dentro de la carpeta del proyecto:

```
cd /var/www/dsw-a13  
npm install
```

Verifica que se haya creado:

```
ls
```

Debe aparecer la carpeta:

```
node_modules
```

Paso 5 – Crear archivo .env

Crea el archivo:

```
nano .env
```

Agrega la configuración:

```
DB_HOST=localhost  
DB_DATABASE=clases  
DB_USER=clases_user  
DB_PASSWORD=TuPasswordSegura123!  
SERVER_PORT=3000  
NODE_ENV=production
```

Guarda con:

```
Ctrl + O  
Enter  
Ctrl + X
```

Paso 6 – Crear la base de datos de la aplicación



Entra a MySQL:

```
mysql -u root -p
```

Crea la base de datos:

```
CREATE DATABASE clases CHARACTER SET utf8mb4;
```

Paso 7 – Crear usuario exclusivo para la aplicación

Dentro de MySQL:

```
CREATE USER 'clases_user'@'localhost' IDENTIFIED BY 'TuPasswordSegura123!';
```

Otorga permisos únicamente sobre esa base:

```
GRANT ALL PRIVILEGES ON clases.* TO 'clases_user'@'localhost';
```

Aplica cambios:

```
FLUSH PRIVILEGES;
```

Verifica permisos:

```
SHOW GRANTS FOR 'clases_user'@'localhost';
```

Sal de MySQL:

```
EXIT;
```

Paso 8 – Importar el script de base de datos

Si tu proyecto incluye `bd.sql`, ejecútalo:

```
mysql -u root -p clases < bd.sql
```

O si el script ya contiene `CREATE DATABASE` y `USE clases`, puedes usar:

```
mysql -u root -p < bd.sql
```

Verifica:

```
mysql -u clases_user -p
```



Dentro de MySQL:

```
USE clases;  
SHOW TABLES;  
SELECT * FROM alumno;  
EXIT;
```

```
[azureuser@vm-dsw-a13-2-ErikaMeneses:/var/www/dsw-a13$ mysql -u clases_user -p  
[Enter password:  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 27  
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2026, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
[mysql> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| clases   |  
| information_schema |  
| performance_schema |  
+-----+  
3 rows in set (0.00 sec)  
  
[mysql> USE clases;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A  
  
Database changed  
[mysql> SHOW TABLES;  
+-----+  
| Tables_in_clases |  
+-----+  
| alumno           |  
| materia          |  
+-----+  
2 rows in set (0.00 sec)  
  
[mysql> SELECT * FROM alumno;  
+-----+-----+-----+-----+  
| id | nombre                | promedio | id_materia |  
+-----+-----+-----+-----+  
| 1  | Sofia Martínez López  | 9.2      | 5          |  
| 2  | Miguel Ángel Torres Ruiz | 8.5      | 6          |  
| 3  | Carolina Herrera Gutiérrez | 9.8      | 4          |  
| 4  | Laura González Sánchez | 8        | 7          |  
| 5  | Ricardo Fernández Moreno | 7.8      | 6          |  
| 6  | Ana María Sánchez Martín | 9.5      | 1          |  
| 7  | Juan Carlos Navarro Jiménez | 6.9      | 1          |  
+-----+-----+-----+-----+
```

Paso 9 – Revisar el archivo principal index.js

Modifica el archivo agregando lo siguiente:



```
const PORT = process.env.SERVER_PORT || 3000;

app.listen(PORT, "0.0.0.0", () => {
  console.log(`Aplicación ejecutándose en el puerto ${PORT}`);
});
```

Es importante usar "0.0.0.0" para que la aplicación pueda recibir conexiones externas.

Debe quedar de la siguiente manera:

```
const express = require('express')
const dotenv = require('dotenv')
const app = express()
const PORT = process.env.SERVER_PORT || 3000;

// Primer carga la configuración del archivo .env
// para que esté disponible en el resto de las llamadas
dotenv.config();
//Establece el motor de vistas a ejs
app.set('view engine', 'ejs')

// Rutas
const homeRouter = require('./routes/home')
app.use("/", homeRouter)
app.use('/home', homeRouter)

//Middleware para el manejo de errores (Debe ser el último middleware a utilizar)
const errorHandler = require('./middlewares/errorhandler')
app.use(errorHandler)

// Inicia el servidor web en el puerto SERVER_PORT
app.listen(PORT, "0.0.0.0", () => {
  console.log(`Aplicación ejecutándose en el puerto ${PORT}`);
});
```

Paso 10 – Probar la aplicación manualmente

Ejecuta:

```
node index.js
```

Desde tu equipo local abre:

```
http://IP_PUBLICA:3000
```

Ejemplo:



http://20.120.55.10:3000

Id	Nombre	Materia	Promedio
1	Sofía Martínez López	Desarrollo de Sistemas Web	9.2
2	Miguel Ángel Torres Ruiz	Enrutamiento Avanzado	8.5
3	Carolina Herrera Gutiérrez	Bases de Datos	9.8
4	Laura González Sánchez	Cómputo en la Nube	8
5	Ricardo Fernández Moreno	Enrutamiento Avanzado	7.8
6	Ana María Sánchez Martín	Arquitecturas de Red	9.5
7	Juan Carlos Navarro Jiménez	Arquitecturas de Red	6.9
8	Beatriz Molina Castro	Sistemas Operativos	8.3

Probar operaciones de la aplicación

En el navegador verifica:

- La página principal carga correctamente.
- La vista de listado muestra datos desde MySQL.
- Las rutas funcionan.
- Los estilos se cargan correctamente.
- No aparecen errores en consola o terminal.

Si la aplicación carga correctamente, detén el servidor con:

Ctrl + C